

LLM Benchmark

June 24, 2026

Students:

Simone Migliaccio
Simone Spezia
Alberto Varini

Professors:

Allegra De Filippo
Michele Lombardi

Contents

LLM Benchmark

Introduction

Problems
Classification

Hardware
platform and
Models
selected

Outputs and
Evaluation
Metrics

Conclusion

1 Introduction

2 Problems Classification

3 Hardware platform and Models selected

4 Outputs and Evaluation Metrics

5 Conclusion

Introduction

Goal and Keypoints

LLM
Benchmark

Introduction

Problems
Classification

Hardware
platform and
Models
selected

Outputs and
Evaluation
Metrics

Conclusion

Goal: create a reliable, scalable and user-friendly benchmark to test the planning capabilities of LLMs on planning problems.

Keypoints

- Problems Classification
- Hardware Platform and LLM Models Selected
- Outputs and Evaluation Metrics

General Structure

LLM Benchmark

Introduction

Problems Classification

Hardware platform and Models selected

Outputs and Evaluation Metrics

Conclusion

This slide is used to show the structure of the repo, under the root folder we have:

- **analysis:** contains the material needed to write reports and the report itself.
- **archive:** contains material provided by the professor and a PDDL guide written to help understand key features of the PDDL.
- **Benchmark Framework:** main folder containing the work done to create this benchmark.

LLM Benchmark/

analysis/

├ domain complexity/

├ notebooks/

└ Reports/

archive/

├ Papers/

├ Planning Domains/

├ References/

└ Quick PDDL Guide/

Benchmark Framework/

└ ...

README.md

General Structure II

LLM
Benchmark

Introduction

Problems
Classification

Hardware
platform and
Models
selected

Outputs and
Evaluation
Metrics

Conclusion

Now the Benchmark Framework folder is highlighted, note that to simplify not all the sub folders are shown.

- *evaluators*: code for running evaluation and parsing of outputs
- *Leonardo script*: files to gain access to leonardo booster
- *outputs*: store outputs of benchmark
- *prompts*: prompts written to be feed to LLMs
- *runner*: contains code to run some tests with LLMs on the booster
- *scripts*: contains code necessary for the scoring of the inputs
- *tasks*: problems selected
- *utils*: executable of validator
- *results*: heatmaps showing output metric results
- *run benchmark*: file that runs whole working sequence

Benchmark Framework/

```
evaluators/  
Leonardo script/  
outputs/  
prompts/  
runner/  
scripts/  
tasks/  
utils/  
results/  
run benchmark.py
```

Problems' Classification

Classification Purpose

LLM
Benchmark

Introduction

Problems
Classification

Hardware
platform and
Models
selected

Outputs and
Evaluation
Metrics

Conclusion

Purpose: In order to obtain reliable outputs to be evaluated after we for sure need a way to classify the PDDL problems to be fed to the LLMs chosen, since we had available 20 different problems a easy but effective way to classify them was needed to choose wisely the problems to be used.

The main idea behind this Input Classification develops around two main concepts:

- need of a numerical score representing the difficulty of the problem.
- classify problems based on the type of reasoning to be employed to solve the problem.

Implementation: Raw and Numerical Score

LLM
Benchmark

Introduction

Problems
Classification

Hardware
platform and
Models
selected

Outputs and
Evaluation
Metrics

Conclusion

As stated before the idea of using a raw score to classify the difficulty of the problems came up because a relatively simple and repeatable way to evaluate problems was needed; in order to obtain the score the code in *LLM benchmark/scripts/score_domains_complexity.py* has been designed. The main idea behind the computation of the score is to evaluate for each instance of a PDDL problem the presence or the absence of features that we believed could increment or decrement the difficulty of a problem, specifically:

- N. of Objects
- N of possible Actions
- N. of Goals
- N. of Numeric goal conditions
- N. of Numeric functions
- N. of Numeric preconditions
- N. of Numeric effects
- Has metric
- Topology

A raw score for each instance of a problem is therefore computed and then a normalization between instances of the same domain is employed; then for each domain a summary is presented.

Implementation: Raw and Numerical Score II

LLM
Benchmark

The raw score for each instance is computed using the following formula:

$$\text{RawScore} = 0.45 \cdot \log(1 + \text{Nactions}) + 0.20 \cdot \log(1 + \text{Ngoals}) + 0.20 \cdot \log(1 + \text{num-score}) + 0.15 \cdot \text{topology}$$

where the num-score is computed by:

$$\text{NumScore} = 1.0 \cdot (\text{N of num effects}) + 2.0 \cdot (\text{N of num preconditions}) + 3.0 \cdot (\text{N of num goal conditions}) + 1.0 \cdot (\text{has metric})$$

the whole domain difficulty is the computed starting from the sum of all the raw scores obtained from the instances of the instances and then normalized 0/100.

A simple example of a domain summary with the scores computed for each instance is shown here on the side, the domain chosen is *counters*.

As it can be noticed other datas are added for completion sake, specifically the mean raw score of the instances and the median raw score.

```
{  
  "domain": "counters",  
  "instances": 20,  
  "total_raw_score": 59.765337,  
  "min_raw_score": 1.820528,  
  "median_raw_score": 3.177554,  
  "mean_raw_score": 2.988267,  
  "max_raw_score": 3.679334,  
  "domain_difficulty_0_100": 18.130829  
},
```

Implementation: Reasoning Type

LLM
Benchmark

Introduction

Problems
Classification

Hardware
platform and
Models
selected

Outputs and
Evaluation
Metrics

Conclusion

Another very important feature of PDDL problems is the fact that different problems may require different kind of reasoning to be properly managed; this said we defined 5 different reasoning categories to classify the problems in this sense:

- Numerical optimization
- Temporal planning
- Logistic
- Logic
- Spatial

Using these 5 categories we divided the problems available into these categories, it is worth noticing that many problems might have mixed reasoning types, so we assigned the final reasoning label to a problem based on the type of reasoning required present most within the problem of interest.

In the end both using this classification and the difficulty score we end up obtaining a good classification able to take into account the main feature of the problems but still easy to deal with even for someone that is not familiar with this topic.

Problems Chosen

LLM
Benchmark

Introduction

Problems
Classification

Hardware
platform and
Models
selected

Outputs and
Evaluation
Metrics

Conclusion

To conclude the Problems' Classification chapter it's useful to highlight among all the problems available the selected ones, since the Temporal Planning and Logic reasoning types have just a single related problem each it is better not to include them in the selection hence we selected problems among all the remaining reasoning categories.

We selected 6 problems total: 3 of them are the hardest ones (one for each reasoning categories left) based on the difficulty score of the domains; the other 3 are the easiest ones, one from each reasoning categories left.

Two sets of problems are therefore obtained:

Hard Set:

- Settlersnumeric (N.opt.)
- Rover (Logistic)
- Block Grouping (Spatial)

Easy Set:

- Fo counters (N.opt.)
- Expedition (Logistic)
- Fo Sailing (Spatial)

Hardware platform and Models selected

Hardware Platform

LLM
Benchmark

Introduction

Problems
Classification

Hardware
platform and
Models
selected

Outputs and
Evaluation
Metrics

Conclusion

Now, since a good amount of computational resources is needed to run the LLMs we started looking for some resources and we ended up with the different alternatives:

- Leonardo HPC by CINECA
- NVIDIA API

Both of them provided good amount of computational resources, more than sufficient to run LLMs but the choice ended up being Leonardo HPC simply because with NVIDIA some problems came up since we needed a good internet connection and we didn't always have a clear feedback from the system.

The CINECA provided us a trial project on Leonardo consisting of a Cluster composed by:

- 32 Cores
- 4 x A100 GPUs (64GB each)
- 500GB RAM

Models Selected

LLM Benchmark

Introduction

Problems Classification

Hardware platform and Models selected

Outputs and Evaluation Metrics

Conclusion

Now for the selection of the LLM models we used a specific tool called *whichLLM* and allows the user to obtain a ranking of LLMs that could be executed effectively on the hardware platform on which we call the tool. We therefore obtained the ranking displayed in the picture below, now that we had a better knowledge about what Leonardo could execute we looked for the models on HuggingFace.

Given the ranking we selected the following models:

- qwen
- gpt oss 120B
- llama 4
- gemma 4
- deepseek

Recommended Models					
#	Model	Quant	Fit / VRAM	Speed	Published
1	deepseek-ai/DeepSeek-V4-Flash 284.6B (13.0Ba)	Q6_K	Full GPU 217.6 GB	13.5 tok/s ~	2026-04-22
2	XiaomiMiMo/MiMo-V2.5 310.8B (15.0Ba)	Q6_K	Full GPU 238.1 GB	12.4 tok/s ~	2026-04-27
3	Qwen/Qwen3.6-27B 27.8B	Q6_K	Full GPU 24.4 GB	34.6 tok/s ~	2026-04-21
4	Qwen/Qwen3-235B-A22B 235.1B (22.0Ba)	Q6_K	Full GPU 181.7 GB	16.3 tok/s ~	2025-04-27
5	google/gemma-4-318-it 32.7B	Q6_K	Full GPU 28.6 GB	29.4 tok/s ~	2026-03-11
6	Qwen/Qwen3-Next-80B-A3B-Instruct 81.3B (3.0Ba)	Q8_0	Full GPU 81.8 GB	32.5 tok/s ~	2025-09-09
7	openai/gpt-oss-120b 120.4B (15.1Ba)	Q6_K	Full GPU 92.8 GB	31.9 tok/s ~	2025-08-04
8	MiniMaxAI/MiniMax-M2.5 228.7B (10.0Ba)	Q6_K	Full GPU 175.3 GB	16.8 tok/s ~	2026-02-12
9	Qwen/Qwen3-36B-A3B 36.5B (3.0Ba)	Q8_0	Full GPU 31.6 GB	86.6 tok/s ~	2025-04-27
10	mistralai/Mistral-Large-Instruct-2411 122.6B	Q5_K_M	Full GPU 98.3 GB	9.6 tok/s ~	2024-11-14

Output and Evaluation Metrics

Output Structure

LLM
Benchmark

Introduction

Problems
Classification

Hardware
platform and
Models
selected

Outputs and
Evaluation
Metrics

Conclusion

DA RIEMPIRE

Evaluation Metrics

LLM
Benchmark

Introduction

Problems
Classification

Hardware
platform and
Models
selected

Outputs and
Evaluation
Metrics

Conclusion

DA RIEMPIRE

Evaluation Metrics II

LLM
Benchmark

Introduction

Problems
Classification

Hardware
platform and
Models
selected

Outputs and
Evaluation
Metrics

Conclusion

DA RIEMPIRE

Conclusion

Conclusion

LLM
Benchmark

Introduction

Problems
Classification

Hardware
platform and
Models
selected

Outputs and
Evaluation
Metrics

Conclusion

DA RIEMPIRE